# HealthNexus
# PHC worker module

Full-stack guide: every feature, its files, its key functions and how they talk to each other

Features covered: joining and approval, dashboard, alerts worklist, citizens, citizen record, examinations, schedule and visits, reports and risk map, outbreak signals, advisories, complaints, Nira, profile, settings

Generated 27 October 2026 from the live code. Every path:line link opens VS Code at that line.

Open any line by hand: Ctrl+P, paste the path and line, press Enter.

# Contents


**0.** How to use this guide 3

**1.** Architecture and the worker module map 4

**2.** Foundations for the worker: roles, catchment, badges 6

**3.** Joining: apply, prove employment, get approved 8

**4.** Dashboard 10

**5.** Alerts: the worklist 12

**6.** Citizens directory 14

**7.** Citizen record 15

**8.** Recording an examination 17

**9.** Schedule and visits 18

**10.** Reports and the risk map 20

**11.** Outbreak signals (worker view) 21

**12.** Advisories (worker view) 23

**13.** Complaints about a citizen, and Notifications 24

**14.** Nira for workers 25

**15.** Profile and settings 26

**16.** How the features talk to each other 27

**17.** API index for the worker module 29

**18.** Data model and numbers to know 32

**19.** Questions across the whole worker module 34

# 0 How to use this guide

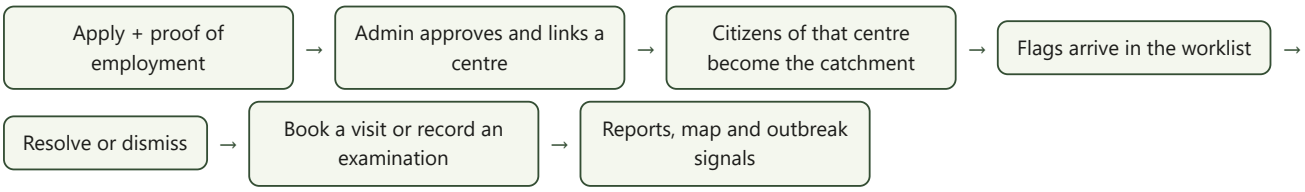
Every function named here has a clickable link that opens VS Code at the exact line.

## Opening any line the examiner asks for

- **Click the blue `path:line` link.** It uses the `vscode://file/…` address, so VS Code opens that file at that line.
- **Or type it.** In VS Code press `Ctrl+P`, paste `backend/app/api/routes/phc.py:374` and press Enter. Every link is written in this form.
- **If a link does nothing,** your PDF viewer is blocking the `vscode://` address; use the Ctrl+P method.
- **Line numbers are real.** They were looked up in the code when this PDF was generated. Regenerate with `python docs/phc_guide/build.py` (needs Chrome or Edge and PyMuPDF); a stale reference stops the build.

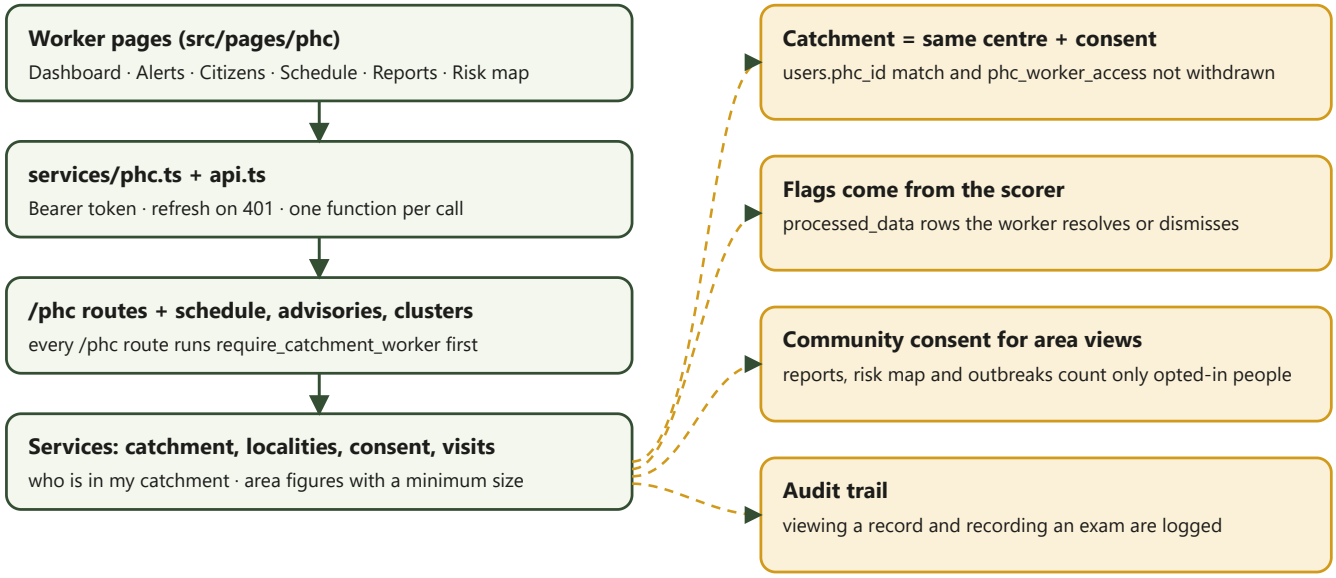
> **Shared with the citizen guide.** Sign-in tokens, `useResource`, error handling, the scoring pipeline and Nira work the same for every role. This guide points to the citizen guide for those and spends its pages on what is **specific to a health worker**: the catchment, the worklist, examinations, visits, area reports and approval.

## The idea in one picture



# 1 Architecture and the worker module map

A worker sees only the citizens linked to their own health centre, and only those who have not withdrawn access.




## The worker module at a glance

| Feature | Page | Frontend service | Backend route | Rules in | Main tables |
|---|---|---|---|---|---|
| Join (apply, documents, approval) | PHCSignup, PendingApproval | auth.ts, documents.ts | routes/auth.py, applications.py, admin.py | accounts.py, documents.py | users, phc_applications, phc_verification_documents |
| Dashboard | phc/PhcDashboard.tsx | phc.ts | routes/phc.py (/dashboard, /overview) | triage.py, catchment.py | processed_data, health_records |
| Alerts (worklist) | phc/PhcAlerts.tsx | phc.ts | routes/phc.py (/worklist, /alerts) | catchment.py | processed_data |
| Citizens | phc/Citizens.tsx | phc.ts | routes/phc.py (/citizens) | catchment.py, localities.py | users, user_profiles |
| Citizen record | phc/CitizenRecord.tsx | phc.ts | routes/phc.py (/citizens/{id}/record) | scoring.py, readings.py | health_records, alerts, scheduled_visits |
| Examination | phc/RecordExam.tsx | phc.ts | routes/phc.py (POST readings) | readings.py | health_records (source = phc) |
| Schedule | phc/Schedule.tsx | phc.ts | routes/schedule.py, phc.py (POST /visits) | visits.py, visit_state.py | scheduled_visits |
| Reports and risk map | phc/Reports.tsx, RiskMap.tsx | phc.ts | routes/phc.py (/reports, /map) | localities.py, consent.py | health_records, users |
| Outbreak signals | Outbreaks.tsx | outbreaks.ts | routes/clusters.py | outbreaks.py | community_data |
| Advisories | Advisories.tsx | advisories.ts | routes/advisories.py | (in the route) | advisories |
| Complaints about a citizen | components/app/ReportCitizen.tsx | account.ts, phc.ts | routes/settings.py, phc.py | complaints.py | concerns |

| Feature | Page | Frontend service | Backend route | Rules in | Main tables |
|---|---|---|---|---|---|
| Nira, profile, settings | Assistant, Profile, Settings | assistant.ts, account.ts | assistant.py, profile.py, settings.py | services/assistant | chat_*, users |

## Technology

Identical to the citizen module: React 18 + TypeScript + Vite + Tailwind v4 in the browser, FastAPI + SQLAlchemy 2 (async) + PostgreSQL behind it, Alembic for schema versions, Recharts and Leaflet for the charts and maps. See the citizen guide, chapter 1, for the diagram and the table.

# 2 Foundations for the worker: roles, catchment, badges

Three ideas run through every worker screen: a worker must be approved, linked to a centre, and only sees consenting citizens of that centre.

## 2.1 The role ladder

Signs up as pending_phc → Admin approves → role phc_worker + phc_id set → require_phc_worker → require_catchment_worker (needs phc_id)

| Function / component | Where | What it does |
|---|---|---|
| `require_phc_worker` | app/core/dependencies.py:58 | 403 unless the account is an approved worker. |
| `require_catchment_worker` | app/services/catchment.py:67 | Approved **and** linked to a facility; otherwise 409. Without it a worker would read '0 flagged' as a quiet day when really no readings can reach them. |
| `phc router` | app/api/routes/phc.py:88 | The guard is attached to the **whole router**, so a new `/phc` route cannot forget it. |
| `ProtectedRoute (worker)` | src/main.tsx:111 | Worker pages are wrapped for role `phc_worker`. |
| `workerState / NoFacility` | src/components/app/phc.tsx:31 | Turns a 409 into a clear 'you are not linked to a facility' screen. |

## 2.2 The catchment

- A citizen is in a worker's catchment when **(1)** `users.phc_id` equals the worker's centre, **(2)** the account is a citizen, and **(3)** the citizen has **not withdrawn** 'health worker access'. Never having answered counts as yes.
- Every worker query starts from `catchment_ids(worker)` or `catchment_citizens(worker)`, so no route can accidentally list another centre's people.
- A citizen outside the catchment answers **the same 404** as one who does not exist (`get_catchment_citizen`), so ids cannot be probed.

| Function / component | Where | What it does |
|---|---|---|
| `catchment_citizens / catchment_ids` | app/services/catchment.py:35 | The one definition of 'my citizens' as a SQL query. |
| `allows_worker_access` | app/services/consent.py:58 | The consent check inside that query (default yes). |
| `in_catchment` | app/services/catchment.py:46 | Same rule for one citizen. |
| `get_catchment_citizen` | app/services/catchment.py:59 | Loads a citizen or answers the uniform 404. |
| `workers_serving` | app/services/catchment.py:74 | How many active workers a centre has; tells a citizen whether anyone reviews their flags. |

## 2.3 The shell: navigation and badges

| Function / component | Where | What it does |
|---|---|---|
| `worker navigation` | src/components/app/AppShell.tsx:203 | Dashboard, Alerts, Citizens, Schedule, Reports, Risk map, Outbreaks, Advisories, Notifications. |
| `loadBadges` | src/components/app/AppShell.tsx:74 | For a worker calls `getPhcBadge`: unread notifications and citizens flagged. |

| Function / component | Where | What it does |
|---|---|---|
| `phc_unread_count` | app/api/routes/alerts.py:78 | Counts distinct citizens with an open flag (the work waiting) and unread notifications. |
| `newsBetween` | src/components/app/AppShell.tsx:87 | When the flagged count **grows** between two polls a toast 'A citizen needs review' appears with the alert sound. |

### Questions you may be asked

**Q. Why attach the guard to the router instead of each route?**
A forgotten dependency on one new route would expose a whole centre's citizens. On the router it is impossible to forget.

**Q. What happens to a worker whose approval is revoked?**
Their role goes back to `pending_phc`, `phc_id` is cleared, their booked visits are cancelled and the citizens told (`revoke` in admin.py). The next request fails the guard.

**Q. Why is 'worker access' default-yes?**
Choosing a centre at onboarding is what links the citizen to it. The consent is the way to withdraw, so only an explicit 'no' removes them from the catchment.

# 3 Joining: apply, prove employment, get approved

A worker applies with their employee details and proof; an administrator decides. Until then the account sees no health information.

| File | Role |
|---|---|
| src/pages/PHCSignup.tsx:6 | The application page; uses the same form as citizens with `variant="phc"`. |
| src/components/auth/SignupForm.tsx:96 | Employee ID, designation, centre name, district, state, years of service. |
| src/pages/PendingApproval.tsx:18 | Progress steps, the proof-of-employment upload, 'Check status'. |
| src/components/app/VerificationDocuments.tsx:58 | Upload list with a status chip per file; the admin's review half is in the same file. |
| app/services/accounts.py:90 | Creates the user (role `pending_phc`) and the `PhcApplication` in one transaction. |
| app/api/routes/applications.py:89 | `POST /phc-application/documents`: validate, store privately, run the checks. |
| app/services/documents.py:117 | Decides auto-passed, flagged or left for manual review. |
| app/api/routes/admin.py:621 | Admin approval: sets the role, links the centre, sends the 'approved' alert. |

| # | Layer | What happens | Open in code |
|---|---|---|---|
| 1 | Browser | The form checks the employee ID pattern (for example TNPHC-CHN-2847) with the same rule the API uses. | src/components/auth/SignupForm.tsx:96 |
| 2 | Router | `POST /auth/register/phc-worker`; a verification email is queued like for a citizen. | app/api/routes/auth.py:91 |
| 3 | Service | One transaction creates the user as **pending_phc** and an application with status `pending`. | app/services/accounts.py:91 |
| 4 | Browser | After verifying the email, `destinationFor` sends a pending account to the approval page. | src/lib/routes.ts:49 |
| 5 | Browser | The worker uploads up to six proofs (ID card, appointment order, registration certificate). | src/components/app/VerificationDocuments.tsx:71 |
| 6 | Router | Only a `pending_phc` applicant may upload; size at most 5 MB; the file's own first bytes must say JPEG, PNG or PDF. | app/api/routes/applications.py:108 |
| 7 | Service | The file is stored in a **private** folder (never served as a public URL) and its SHA-256 is saved. | app/services/documents.py:46 |
| 8 | Service | `check` runs: the same file used by another person's application; image-editor traces in the metadata; low resolution; OCR text searched for the employee ID, the name and the centre. | app/services/documents.py:156 |
| 9 | Service | ID and name both found and nothing suspicious → `auto_passed`; neither found, or a high tamper score → `flagged`; otherwise left `pending` for a person. | app/services/documents.py:146 |
| 10 | Router | The admin reviews the application and its files, then approves with a centre. | app/api/routes/admin.py:621 |
| 11 | Database | Approval sets `role = phc_worker`, `phc_id`, district and state and writes an audit entry; the worker gets an alert 'Your PHC worker account is approved'. | app/api/routes/admin.py:648 |
| 12 | Browser | 'Check status' refreshes the user; a worker now lands on the worker dashboard. | src/pages/PendingApproval.tsx:34 |

> The automatic checks **assist the administrator, they do not decide**. 'Auto-passed' is a hint on the admin's screen; only an administrator can approve. Documents are opened by the admin through a logged endpoint, so every view is traceable.

### Questions you may be asked

**Q. Why not let anyone who types an employee ID in?**
A worker can read health records. Approval by a person with evidence is the control, and the account has no access to anyone's data until then.

**Q. How do you stop a forged or reused document?**
Duplicate detection by SHA-256 across applicants, image-editor metadata, resolution, and OCR that must find the employee ID and name. Anything doubtful is flagged for the admin.

**Q. Can a worker upload documents after approval?**
No. `upload` and `remove` return 409 once the application is decided.

**Q. Where are the files kept?**
In `private_uploads`, outside the public media folder, under generated names.

# 4 Dashboard

The start-of-day view: how many people need review, how many readings arrived, what is due today, and eight weeks of trend.

| File | Role |
|---|---|
| src/pages/phc/PhcDashboard.tsx:66 | Greeting, quick actions, counts, severity filter, the first six people to review, four KPI charts. |
| src/services/phc.ts:187 | `getDashboard`, `getOverview`, `getWorklist`. |
| app/api/routes/phc.py:181 | `GET /phc/dashboard`: the counts. |
| app/api/routes/phc.py:233 | `GET /phc/overview`: four KPIs by calendar week. |
| app/services/triage.py:17 | The severity bands; the dashboard's SQL uses the **same list**, so the two cannot drift. |
| src/components/app/phc.tsx:132 | One person's open flags as a card, with Resolve, Dismiss, Book visit and Record. |

| Function / component | Where | What it does |
|---|---|---|
| `dashboard` | app/api/routes/phc.py:181 | Citizens in catchment; citizens flagged; open flags by severity; readings and people submitting in the last 7 days; engagement counts; visits needing an outcome; visits later today. |
| `_severity_sql` | app/api/routes/phc.py:102 | Builds a SQL `CASE` from `BANDS` so the count by severity matches the severity shown on every card. |
| `_directory` | app/api/routes/phc.py:137 | Joins each citizen with reading and open-flag aggregates and derives the engagement status (shared with the Citizens page). |
| `overview` | app/api/routes/phc.py:233 | Readings submitted, citizens who submitted, flags raised, flags cleared; eight Monday-start weeks in India time; the current week marked partial. |
| `KpiCard` | src/pages/phc/PhcDashboard.tsx:18 | Last full week as the headline, this week so far beneath, a bar per week (the partial one is paler). |

| # | Layer | What happens | Open in code |
|---|---|---|---|
| 1 | Browser | Three requests start together: counts, KPI series and the worklist. | src/pages/phc/PhcDashboard.tsx:70 |
| 2 | Guard | `require_catchment_worker` runs for the whole `/phc` router; a worker with no centre gets 409 and the page shows the 'not linked' screen. | app/services/catchment.py:67 |
| 3 | Database | Open flags are rows of `processed_data` with `is_anomaly` true and no `resolution`, restricted to `catchment_ids(worker)`. | app/api/routes/phc.py:98 |
| 4 | Service | Engagement status per citizen: **attention** (open flag), **never logged**, **silent** (no reading for 14 days), otherwise **ok**. | app/api/routes/phc.py:160 |
| 5 | Browser | The severity tabs filter the worklist in the browser; the URL keeps the choice. | src/pages/phc/PhcDashboard.tsx:69 |
| 6 | Browser | After a card is resolved the page reloads both the worklist and the counts. | src/pages/phc/PhcDashboard.tsx:88 |

### Questions you may be asked

**Q. Why are weeks counted in India time and from Monday?**
A flag raised at 11 pm local would otherwise land in the next day's bucket in UTC. The SQL converts to the local zone before truncating to the week.

**Q. What is a 'silent' citizen?**
Someone whose last reading is older than 14 days (`SILENT_AFTER`); a person who joined and never logged is 'never logged', which ranks above silent.

# 5 Alerts: the worklist

Flags arrive per reading but are worked **per person**: one citizen is one card, however many flags they have.

| File | Role |
|---|---|
| src/pages/phc/PhcAlerts.tsx:55 | Tabs Needs review and Recent activity; search; time, severity and locality filters; cards and pop-ups. |
| src/components/app/compact.tsx:52 | The green **Resolve** and red **Dismiss** buttons, with the dismissal reason. |
| app/api/routes/phc.py:362 | `GET /phc/worklist`: open flags grouped by citizen. |
| app/api/routes/phc.py:453 | `POST /phc/worklist/citizens/{id}/resolve`: clear all of one person's flags. |
| app/api/routes/phc.py:336 | `POST /phc/queue/{id}/seen`: clear one flag. |
| app/api/routes/phc.py:725 | `GET /phc/alerts`: recent flags, open and cleared, for the Recent activity tab. |
| app/schemas/phc.py:88 | The decision: reviewed, or dismissed **with** a reason. |

| Function / component | Where | What it does |
|---|---|---|
| `worklist` | app/api/routes/phc.py:362 | Groups open flags by citizen (count, worst score, first and latest time), orders worst first, attaches up to 10 worst flags per person and the worker's next visit with them. |
| `_search` | app/api/routes/phc.py:112 | A name, or an `HN-1A2B3C4D` reference matched against the start of the citizen's id. |
| `_apply` | app/api/routes/phc.py:325 | The one update used by both resolve routes: who, when, the resolution, the reason and note. |
| `Resolve._reason_matches` | app/schemas/phc.py:94 | A dismissal must carry a reason; a plain review must not. |
| `PhcAlerts` | src/pages/phc/PhcAlerts.tsx:55 | Filters run in the browser over the loaded list; cards show name, ID, place, how long it has been open and the severity. |
| `WorkDetails` | src/pages/phc/PhcAlerts.tsx:219 | The pop-up: details, the flags newest first (click one to resolve just that), Resolve all, Dismiss, Book visit, Full record. |

| # | Layer | What happens | Open in code |
|---|---|---|---|
| 1 | Browser | The page loads the worklist once and filters locally by search, period, severity and locality. | src/pages/phc/PhcAlerts.tsx:65 |
| 2 | Router | `GROUP BY user_id` over open flags in the catchment; ordered by worst score, then latest. | app/api/routes/phc.py:370 |
| 3 | Service | Each person's flags are capped at 10 (worst first) so one noisy citizen cannot crowd out the list. | app/api/routes/phc.py:94 |
| 4 | Browser | A card opens a pop-up; clicking a single flag opens that flag on its own. | src/pages/phc/PhcAlerts.tsx:224 |
| 5 | Browser | **Resolve** (green) marks reviewed; **Dismiss** (red) asks why: false alarm, already being treated, contacted with no action needed, or other. | src/components/app/compact.tsx:45 |
| 6 | Router | `resolve_citizen` updates every open flag of that citizen in one statement; `mark_seen` locks and updates one (409 if it was already cleared). | app/api/routes/phc.py:460 |
| 7 | Database | `processed_data` gets `resolution`, `reviewed_by`, `reviewed_at`, `dismiss_reason` and the note. | app/api/routes/phc.py:327 |
| 8 | Browser | Booking a visit from the pop-up opens the visit form with the citizen already chosen and the flag attached. | src/pages/phc/PhcAlerts.tsx:230 |

**Effects elsewhere:** the citizen's dashboard open-flag count falls and their health score rises; their History marks the reading 'reviewed'; the badge on the worker's Alerts menu drops on the next poll.

### Questions you may be asked

**Q. Why group by person?**

Five flags for one citizen are one problem to follow up, not five. The card shows the worst and lets the worker clear all or one.

**Q. Why must a dismissal carry a reason?**

A dismissal without a reason cannot be audited. The reason is stored on the flag (`dismiss_reason`) and the note beside it.

**Q. Can a worker clear another centre's flag?**

No. Both routes filter on `catchment_ids(worker)`; a foreign flag is 404 'Case not found'.

# 6 Citizens directory

Everyone in the catchment, sorted by what is worth opening first, with the actions a worker needs on each row.

| File | Role |
|---|---|
| src/pages/phc/Citizens.tsx:29 | Search, status and locality filters; numbered table with coloured status; View, Book and Examine on each row. |
| app/api/routes/phc.py:468 | `GET /phc/citizens`. |
| app/services/localities.py:24 | Blank areas become 'Unspecified'. |

| Status | Meaning | Colour |
|---|---|---|
| Needs attention | has at least one open flag | red |
| Never logged | joined but no reading yet | violet |
| Gone quiet | last reading more than 14 days ago | amber |
| Logging | recent readings, nothing open | green |

| Function / component | Where | What it does |
|---|---|---|
| `citizens` | app/api/routes/phc.py:468 | Filters by name or HN reference, locality and status; ranks attention, never logged, silent, ok; within a group the worst flag first, then the longest silent. |
| `CitizenDetails` | src/pages/phc/Citizens.tsx:169 | Pop-up: reference, locality, tap-to-call phone, age, gender, counts, last reading, joined; Book visit, Record examination, Full record, Report a problem. |
| `StatusPill` | src/pages/phc/Citizens.tsx:25 | The coloured status. |

| # | Layer | What happens | Open in code |
|---|---|---|---|
| 1 | Browser | Typing in search waits 300 ms before asking, so each keystroke is not a request. | src/pages/phc/Citizens.tsx:40 |
| 2 | Router | The query joins the catchment with per-citizen reading and flag aggregates and applies the three filters. | app/api/routes/phc.py:479 |
| 3 | Database | Paged with `limit` 50 and `offset`; the total and the per-status counts are computed over the **whole** catchment, not just the page. | app/api/routes/phc.py:523 |
| 4 | Browser | Row actions: View opens the pop-up; Book opens the visit form; Examine goes to the examination page. | src/pages/phc/Citizens.tsx:127 |

### Questions you may be asked

**Q. Why compute counts over the whole catchment?**

A page of 50 cannot say how many are silent. The status dropdown shows the true counts next to each option.

**Q. Can the worker see citizens who withdrew access?**

No. They are not in `catchment_citizens`, so they are absent here and in every other worker view.

# 7 Citizen record

Everything about one citizen in one request, with the model's reasoning for each flag. Every view is written to the audit log.

| File | Role |
|---|---|
| src/pages/phc/CitizenRecord.tsx:21 | Header actions, period selector, trend chart, Submissions, Visits, Alerts sent, pop-ups. |
| app/api/routes/phc.py:582 | `GET /phc/citizens/{id}/record?days=`. |
| src/components/app/ModelInsights.tsx:27 | Per-vital explanation of a flag, worded for a worker. |
| src/components/app/compact.tsx:11 | The one-line rows used by Submissions and Alerts sent. |
| src/components/app/ReportCitizen.tsx:12 | 'Report a problem' about this citizen (chapter 13). |

| Function / component | Where | What it does |
|---|---|---|
| `citizen_record` | app/api/routes/phc.py:582 | Identity and profile, latest value of each vital with the citizen's own baseline and z-score, a timeline, alerts sent, past and upcoming visits, and flags still open from before the window. |
| `_entry` | app/api/routes/phc.py:536 | One timeline row: values, symptoms, severity, the signal sentence, the resolution, and the stored per-vital working ( `checks` ). |
| `ScoreBreakdown` | src/components/app/ModelInsights.tsx:27 | Shows each vital's value against the range and the citizen's baseline, and what it added to the flag. |
| `PopulationNote` | src/components/app/ModelInsights.tsx:75 | The Isolation Forest's view (how unusual next to 71 people); informs only. |

| # | Layer | What happens | Open in code |
|---|---|---|---|
| 1 | Browser | The record and the vitals vocabulary load together; the period (30, 90, 180, 365 days) lives in the URL. | src/pages/phc/CitizenRecord.tsx:26 |
| 2 | Guard | `get_catchment_citizen`: a citizen outside the catchment is the same 404 as an unknown id. | app/services/catchment.py:59 |
| 3 | Database | Readings of the window (up to 2000) joined to their scores; alerts (50), visits (50 past, 20 upcoming). | app/api/routes/phc.py:595 |
| 4 | Service | Flags still open from **before** the window are counted separately and never mixed into the window's numbers. | app/api/routes/phc.py:651 |
| 5 | Database | An audit row `view_citizen_record` is written with the worker and the IP, in the same request. | app/api/routes/phc.py:671 |
| 6 | Browser | Submissions shows topic, time and severity per reading; opening one shows every vital, the symptoms and **why it was flagged**, with Resolve and Dismiss if it is still open. | src/pages/phc/CitizenRecord.tsx:259 |

### Questions you may be asked

**Q. Why log a worker viewing a record?**

It is sensitive personal data. The log lets a centre or an administrator answer 'who looked at this person's record and when'.

**Q. Why separate 'from before this period'?**
A window of 30 days must not hide a flag that has been open for 40; it is reported in a notice beside the window instead of inside its counts.

# 8 Recording an examination

When a worker measures a citizen (blood pressure, sugar and so on) the reading is stored under the citizen, marked as theirs, and scored like any other.

| File | Role |
|---|---|
| src/pages/phc/RecordExam.tsx:42 | Form with a field per measurement, symptoms, place and optional earlier time; the result replaces the form. |
| src/services/phc.ts:239 | `POST /phc/citizens/{id}/readings`. |
| app/api/routes/phc.py:560 | Checks, stores and scores the reading; writes the audit row. |
| app/schemas/health.py:33 | Request rules shared with every reading: known vitals, plausible ranges, systolic above diastolic, up to 12 symptoms, time not in the future or more than 30 days back. |
| app/services/readings.py:145 | Store, score, alert, email, commit: the same function the watch uses. |

| # | Layer | What happens | Open in code |
|---|---|---|---|
| 1 | Browser | Each value is checked as it is typed: it must be a number, inside the plausible range, and diastolic must be below systolic. | src/pages/phc/RecordExam.tsx:25 |
| 2 | Browser | A place is **required**; the checkbox 'I measured this earlier' lets the worker set a past time. | src/pages/phc/RecordExam.tsx:109 |
| 3 | Router | `POST /phc/citizens/{id}/readings`: the guard, then `get_catchment_citizen` (uniform 404 outside the catchment). | app/api/routes/phc.py:560 |
| 4 | Router | If the citizen turned health tracking off, nothing is stored (409). | app/api/routes/phc.py:569 |
| 5 | Service | `submit_reading(source='phc', recorded_by=worker, location=…)` scores it against the citizen's own baseline and creates the alert and emails if flagged. | app/api/routes/phc.py:573 |
| 6 | Database | An audit row `record_examination` links the worker to the new reading. | app/api/routes/phc.py:575 |
| 7 | Browser | The result card replaces the form (so a second submit cannot create a duplicate) and plays the alert or clear sound. | src/pages/phc/RecordExam.tsx:120 |

> **What the citizen sees:** the reading appears in their History with 'recorded by <worker>, <place>', cannot be deleted by them (only the centre can change it), and if flagged they get the same alert and email as for a watch reading.

### Questions you may be asked

**Q. Why does a watch not provide blood pressure?**

Home watches measure heart rate, $SpO_2$, breathing rate and temperature. Pressure and sugar need a cuff or meter, so they come from a worker's examination.

**Q. Can a worker back-date an examination?**

Up to 30 days, never into the future (`_recent_past`), and the true storage time is kept in `created_at`.

# 9 Schedule and visits

A worker books, edits, completes or cancels visits; the citizen is told each time.

| File | Role |
|---|---|
| src/pages/phc/Schedule.tsx:30 | Tabs Needs outcome, Upcoming, Done, Missed, Cancelled; search; kind and period filters; cards and pop-up. |
| src/pages/phc/Schedule.tsx:248 | The booking and editing form: citizen, date, time, length, kind, place, reason, private notes. |
| app/api/routes/phc.py:858 | `POST /phc/visits`. |
| app/api/routes/schedule.py:262 | `GET /schedule/all`: visits of the last 180 days onward. |
| app/api/routes/schedule.py:163 | `PATCH /schedule/{id}`: complete, nobody there, cancel, move, edit. |
| app/services/visits.py:24 | Alerts and emails to both people when a visit is booked, changed or has no outcome. |
| app/services/visit_state.py:42 | Upcoming, today, due now, overdue: derived from the status and the clock, never stored. |

| Action | Allowed when | Effect |
|---|---|---|
| Book | citizen in catchment; time in the future and within 90 days; the citizen has no other open visit | Creates the visit; the citizen's alert and email; if made from a flag, that flag is marked reviewed ('Follow-up visit booked'). |
| Mark done | the visit is today or its time has passed (not for a later day; move it first if it happened early) | status = completed. |
| Nobody there | once the booked time has passed | status = missed (counts toward a complaint if unexplained, chapter 13). |
| Move or edit time | more than 8 hours before it starts | New time; the citizen is notified. |
| Edit other details | any time while booked | Kind, length, place, reason, private notes. |
| Cancel | more than 8 hours ahead, with a reason of 10+ characters | status = cancelled; the citizen sees the reason. |

| # | Layer | What happens | Open in code |
|---|---|---|---|
| 1 | Browser | 'Book a visit' (or Book on a card) opens `VisitForm`; the citizen can be searched by name or HN reference. | src/pages/phc/Schedule.tsx:265 |
| 2 | Router | `book_visit` loads the citizen with `get_catchment_citizen` and checks the time. | app/api/routes/phc.py:862 |
| 3 | Router | One open visit per citizen: a second booking is refused with 'Move that visit instead'. | app/api/routes/phc.py:874 |
| 4 | Service | `notify_booked` writes the citizen's alert (with the sound on their next poll) and the worker's, and queues both emails if their preferences allow. | app/services/visits.py:24 |
| 5 | Browser | A success message confirms the booking for the worker. | src/pages/phc/Schedule.tsx:293 |
| 6 | Router | `update_visit` locks the row, refuses a visit that already has an outcome, and applies the action's rule. | app/api/routes/schedule.py:182 |
| 7 | Router | Cancel and time changes call `check_cutoff` (8 hours) on the server; the page also hides the buttons. | app/api/routes/schedule.py:36 |
| 8 | Service | `notify_changed` tells the citizen what happened, including the reason. | app/services/visits.py:40 |

### Questions you may be asked

**Q. Why is 'overdue' not stored?**

It depends on the clock. Storing it would need a job to keep it right; deriving it with `visit_state` is always correct and cannot drift.

**Q. What stops two workers double-booking the same citizen?**

The check for an existing open visit, plus the citizen being in only one catchment.

**Q. Who sees the private notes?**

Only the worker. `VisitOut.notes` is set in `visit_for_worker` and never in the citizen's view.

# 10 Reports and the risk map

Area-level views for the centre. They count only citizens who opted into community monitoring, and never show a group smaller than three.

| File | Role |
|---|---|
| src/pages/phc/Reports.tsx:12 | Totals, a daily bar chart of readings and flags, and a locality table. |
| src/pages/phc/RiskMap.tsx:15 | Leaflet map of localities coloured by risk, with a ranked list beside it. |
| app/api/routes/phc.py:766 | `GET /phc/reports?days=` (7 to 180, default 30). |
| app/api/routes/phc.py:845 | `GET /phc/map` and `/phc/locality-risk`. |
| app/services/localities.py:51 | Ranks localities by the share of citizens with an open flag. |
| app/services/localities.py:36 | Folds any locality under 3 citizens into 'Other localities'; drops that too if still under 3. |
| app/services/consent.py:67 | The consent filter (default **no**) every area-level query includes. |

| Risk level | Share of the locality's citizens with an open flag |
|---|---|
| High | 30% or more |
| Elevated | 15% to under 30% |
| Some | above 0% |
| None | 0% |

| # | Layer | What happens | Open in code |
|---|---|---|---|
| 1 | Browser | The period selector changes `days`; the URL keeps it. | src/pages/phc/Reports.tsx:15 |
| 2 | Router | Whole local days: 'last 30 days' is today and the 29 before it, counted in India time. | app/api/routes/phc.py:773 |
| 3 | Database | Totals and the daily series use the whole catchment; the **locality table** adds `counts_in_community`. | app/api/routes/phc.py:802 |
| 4 | Service | `suppress` removes small cells so a locality of one or two people cannot be identified. | app/services/localities.py:36 |
| 5 | Browser | The map plots each locality at the average position of its citizens; colour is the risk level. | src/pages/phc/RiskMap.tsx:34 |

### Questions you may be asked

**Q. Why show totals for everyone but localities only for opted-in people?**

A total says nothing about a person. A small locality does: the numbers could point at one household, so it needs both consent and a minimum size.

**Q. What does 'Other localities' mean?**

Localities under three people merged together, so the rest of the table is still complete but no small group is exposed.

# 11 Outbreak signals (worker view)

The system looks for clusters of people with the same illness close together in space and time, and asks the worker to confirm or reject each one.

| File | Role |
|---|---|
| src/pages/Outbreaks.tsx:68 | Full-width map, a searchable signals table, a pop-up with the reasons and a feedback form. |
| app/api/routes/clusters.py:155 | `GET /clusters`: for a worker, only clusters that include their centre's citizens. |
| app/api/routes/clusters.py:77 | Writes the plain-language reasons with the real numbers. |
| app/services/outbreaks.py:48 | One clustering run: collect cases, cluster, test against the usual level, store. |
| model/healthnexus_ml/clustering.py:79 | Space-time DBSCAN. |
| app/services/outbreaks.py:113 | Recognises the same signal across runs, so 'seen' and feedback carry over. |

| Rule | Value |
|---|---|
| Cases | a reading with fever, body ache or rash (or 38.0 °C and above); cough, breathlessness or chest pain (or $SpO_2$ 93 or below); diarrhoea, nausea or thirst |
| Window | the last 14 days; one point per person (their first onset) |
| Cluster | at least 3 different people within 1.5 km, where one day apart counts as 0.5 km |
| Unusual? | compared with the same area in the 56 days before; flagged as a possible outbreak when the chance of this many cases is below 1% (Poisson test) |
| Who counts | only active citizens who opted into community monitoring |
| How often | every 6 hours in the background; an administrator can run it now |

| # | Layer | What happens | Open in code |
|---|---|---|---|
| 1 | Background | A task started at server start runs the clustering every 6 hours. | app/services/outbreaks.py:139 |
| 2 | Service | Cases come from consenting citizens' readings with a location; each syndrome is clustered separately. | app/services/outbreaks.py:74 |
| 3 | Database | Each cluster is stored with its cases, expected count, p-value, centre, radius, localities and the centres involved (`phc_ids`). | app/services/outbreaks.py:93 |
| 4 | Router | A worker's list is filtered to clusters whose `phc_ids` contain their centre. | app/api/routes/clusters.py:123 |
| 5 | Browser | Opening a signal shows the numbered reasons: how many people, the usual level, the p-value, symptoms and the share of nearby residents. | src/pages/Outbreaks.tsx:246 |
| 6 | Router | 'Mark as seen' records who and when; the feedback (real outbreak, false alarm, unsure) keeps a track record of the clustering's accuracy. | app/api/routes/clusters.py:185 |
| 7 | Browser | 'Write an advisory' opens the advisory form pre-filled from the signal (chapter 12). | src/pages/Outbreaks.tsx:340 |

### Questions you may be asked

**Q. Is this a diagnosis?**

No. It says 'more people than usual with similar symptoms live close together'. The worker decides, and their feedback is stored so the signal's accuracy can be measured.

**Q. Why a 56-day baseline?**
It estimates how many cases the same area usually has; the run compares the current count with that expected number (never below 0.5).

# 12 Advisories (worker view)

A worker can warn the citizens of their own centre, and the people living near it. Red is allowed only when the data supports it.

| File | Role |
|---|---|
| src/pages/Advisories.tsx:80 | A composer, then the history table with search and filters and a details pop-up. |
| src/pages/Advisories.tsx:246 | Chooses who gets it: for a worker it is their own centre, optionally plus people living within a radius. |
| app/api/routes/advisories.py:368 | Publishes after the red check and the target resolution. |
| app/api/routes/advisories.py:256 | A worker is forced to `scope = centre` for their own centre. |
| app/api/routes/advisories.py:161 | Decides whether red (emergency) is justified. |
| app/api/routes/advisories.py:68 | Which citizens see which advisory. |

| Colour | Meaning | Who can choose it |
|---|---|---|
| Green | for information | anyone |
| Yellow | take precautions | anyone (default) |
| Red | emergency, act now | only if a flagged outbreak with 10 or more cases exists, or at least 15 citizens (5% of the area) had high alerts this week, and no other red is running; a reason of 30+ characters is required |

| # | Layer | What happens | Open in code |
|---|---|---|---|
| 1 | Browser | Choosing red asks the server first; if refused the choice snaps back to yellow with the real numbers. | src/pages/Advisories.tsx:124 |
| 2 | Router | `red_check` counts flagged outbreaks, citizens with high alerts in 7 days, and running red advisories. | app/api/routes/advisories.py:155 |
| 3 | Router | `create_advisory` repeats the red check (a client cannot skip it) and requires the written reason. | app/api/routes/advisories.py:371 |
| 4 | Service | `resolve_target` gives a worker their own centre and, if asked, the radius around it; the reach is previewed beforehand. | app/api/routes/advisories.py:349 |
| 5 | Database | The advisory is stored with its scope, radius and the centre's coordinates. | app/api/routes/advisories.py:382 |
| 6 | Browser | Citizens matching `targeting` see it on their Alerts page; the worker sees it in the history and can withdraw it or record whether it helped. | app/api/routes/advisories.py:416 |

### Questions you may be asked

**Q. Why restrict red?**

A false red alarm causes panic and teaches people to ignore alerts. The server checks outbreak size and the share of high alerts before allowing it.

**Q. Can a worker advise another centre's citizens?**

No: for a worker `resolve_target` rejects any scope except their own centre.

# 13 Complaints about a citizen, and Notifications

A worker can report repeated missed visits only when the record proves it; their own notifications have a page of their own.

| File | Role |
|---|---|
| src/components/app/ReportCitizen.tsx:12 | 'Report a problem': keeps missing visits, or behaviour during a visit. |
| app/services/complaints.py:56 | Counts the citizen's missed visits at this centre in 90 days and how many are unexplained. |
| app/services/complaints.py:119 | Refuses a missed-visits complaint without evidence; attaches the visits as evidence. |
| app/api/routes/phc.py:934 | `GET /phc/citizens/{id}/missed-visits`: the evidence, before the worker writes anything. |
| src/pages/Notifications.tsx:6 | The worker's own alerts: bookings, changes, visits waiting for an outcome. |
| app/api/routes/alerts.py:30 | The same endpoint the citizen uses, for the worker's own alerts. |

| # | Layer | What happens | Open in code |
|---|---|---|---|
| 1 | Browser | 'Report a problem' on a citizen's pop-up or record shows the evidence first. | src/components/app/ReportCitizen.tsx:13 |
| 2 | Service | The rule: at least **3 missed visits with no reason given** in the last 90 days. A visit the citizen explained does not count. | app/services/complaints.py:22 |
| 3 | Browser | 'Send to admin' stays disabled until the rule is met. | src/components/app/ReportCitizen.tsx:90 |
| 4 | Router | `POST /settings/concerns` calls `prepare`, which checks the rule **again** and stores the visits as evidence on the complaint. | app/api/routes/settings.py:354 |
| 5 | Browser | The district administrator sees it under 'From PHC workers' with the evidence, and decides. | src/pages/admin/Concerns.tsx:202 |

### Questions you may be asked

**Q. Why must a citizen's explanation stop a complaint?**
Missing a visit while in hospital is not misconduct. The citizen can explain a missed visit from their Visits page, and an explained one stops counting.

**Q. Can the rule be bypassed from the browser?**
No. The API raises 422 with the exact count if fewer than three are unexplained.

# 14 Nira for workers

The same assistant pipeline as for citizens, with the catchment's tools and the worker's centre bound from the sign-in.

The flow is exactly the one in the citizen guide (chapter 11): safety check, plan, read-only retrieval, wording, number check, storage. What differs is **which queries are open** and **what is bound**.

| Function / component | Where | What it does |
|---|---|---|
| `tools_for` | app/services/assistant/catalog.py:740 | A worker gets 11 of the 29 queries: own alerts, catchment overview, flagged citizens, citizen summary, silent citizens, own schedule, locality breakdown, catchment trend, symptom counts, advisories published, outbreak signals. |
| `_binds` | app/services/assistant/__init__.py:131 | Adds `:me`, `:phc_id` and `:district` from the signed-in user, never from the question or the model. |
| `needs_facility` | app/services/assistant/__init__.py:242 | A worker with no centre gets the 'not linked' message instead of an empty answer. |

- Area-level queries (locality breakdown, trend, symptom counts) include only citizens who opted into community monitoring, the same rule as the Reports page.
- Because the catchment is bound as `:phc_id`, asking about 'my citizens' can never reach another centre.

### Questions you may be asked

**Q. Can Nira show an individual citizen's readings to a worker?**

Only through the `citizen_summary` query, restricted to the catchment and to citizens who have not withdrawn worker access.

# 15 Profile and settings

The same pages and endpoints as for a citizen, with a few worker-only fields.

The profile, password, sessions and email-change flows are identical to the citizen guide (chapters 3, 12 and 13). The differences:

| Where | Worker difference | Code |
|---|---|---|
| Profile | A **department** field is editable (hidden for citizens). | src/pages/Profile.tsx:76 |
| Settings → tabs | 'Your work' replaces 'Your health centre': facility, designation and working hours. | src/pages/Settings.tsx:455 |
| Working hours | Stored in preferences; the **default time** offered when booking a visit is the start of the working day. | src/components/app/phc.tsx:216 |
| Preferences | Email choices for alerts and visits and the least severe flag worth an email apply to a worker too; language is a citizen-only choice. | app/services/preferences.py:89 |
| Privacy tab | Shows only the consents that apply to a worker; the three citizen ones are hidden. | src/pages/Settings.tsx:230 |
| Support | A worker raises app and access problems here; complaints about citizens start from the citizen's record. | src/pages/Settings.tsx:306 |

| Function / component | Where | What it does |
|---|---|---|
| `update_profile` | app/api/routes/profile.py:121 | Same route as citizens; `department` is accepted for any role but only workers see it. |
| `update_preferences` | app/api/routes/settings.py:53 | `working_hours` is validated: the end must be after the start. |
| `reading_flagged (worker part)` | app/services/notify.py:65 | A worker is emailed about a new flag at their centre only if the flag is reviewable and their own preferences and minimum severity allow it. |

### Questions you may be asked

**Q. Does a worker choose a centre in settings?**

No. The centre is set by the administrator at approval; a worker without one is shown the 'not linked' screen and no readings reach them.

# 16 How the features talk to each other

Features never call each other's pages. They share **tables and services**: one feature writes a row, another reads it.

## The shared spine

processed_data (flags) → health_records → scheduled_visits → alerts → consent_records → phc_applications

| When this happens | Written by | Stored in | Then these react |
|---|---|---|---|
| A citizen's reading is flagged | app/services/readings.py:78 | processed_data; alerts; email outbox | The citizen appears in the worklist · the Alerts badge grows within 30 s with a toast and sound · emails to workers who asked (`reading_flagged`) |
| Worker resolves or dismisses | app/api/routes/phc.py:325 | processed_data.resolution | The citizen's dashboard shows fewer open flags · History marks it reviewed · the badge drops |
| Worker books a visit | app/api/routes/phc.py:858 | scheduled_visits; alerts; email | The citizen's Visits and Alerts pages and next-visit line · the linked flag is marked reviewed |
| Citizen cancels a visit | app/api/routes/schedule.py:235 | scheduled_visits | The worker's card shows 'cancelled by the citizen' with the reason · worker alert and email |
| Visit time passes with no outcome | app/services/visits.py:56 | alerts (once) | Both alerted; the worker's 'Needs outcome' tab lists it; Mark done or Nobody there |
| Citizen explains a missed visit | app/api/routes/schedule.py:212 | scheduled_visits.missed_reason | The worker sees the reason · the visit stops counting toward a complaint |
| Worker records an examination | app/api/routes/phc.py:560 | health_records (source phc) | Scored like any reading · shows in the citizen's History as the worker's · may raise a flag, alert and email |
| Citizen withdraws worker access | app/services/consent.py:87 | consent_records | They vanish from every worker page · their open visits are cancelled |
| Citizen changes health centre | app/api/routes/settings.py:443 | users.phc_id | Moves from one catchment to another · the old centre's visits are cancelled |
| Admin approves a worker | app/api/routes/admin.py:621 | users.role, users.phc_id | The next request passes `require_phc_worker`; the catchment appears |
| Outbreak run finds a cluster | app/services/outbreaks.py:48 | community_data | It lists on the Outbreaks page of every worker whose centre is involved |
| Worker publishes an advisory | app/api/routes/advisories.py:368 | advisories | Matching citizens see it on their Alerts page |
| Worker reports a citizen | app/api/routes/settings.py:349 | concerns (with evidence) | The district administrator's Complaints page; the reply comes back to the worker |

## Two journeys end to end

### A. From a flagged reading to a visit

Citizen's watch sync → score_reading flags → Alert + worker email → Worklist card → Worker opens details → Book visit → Citizen toast + sound → Visit on the citizen's page

### B. From a new application to a working catchment

Apply + proof → OCR and tamper checks → Admin reviews → approve: role + phc_id → ProtectedRoute lets them in → Citizens of the centre appear

> **If the examiner asks 'how do two features communicate?'** one feature writes a row through a service function; the other reads that table through its own route. There is no feature-to-feature call, which is why a page can be changed without touching another.

# 17 API index for the worker module

Every endpoint the worker's screens use, generated by scanning the route files, with the front-end function that calls it.

| Endpoint | Handler | Called from |
|---|---|---|
| **POST** `/auth/register/phc-worker` | app/api/routes/auth.py:91 | src/services/auth.ts:81 |
| **POST** `/auth/login` | app/api/routes/auth.py:100 | src/services/auth.ts:84 |
| **POST** `/auth/refresh` | app/api/routes/auth.py:111 | src/services/api.ts:84 |
| **POST** `/auth/logout` | app/api/routes/auth.py:131 | src/services/auth.ts:93 |
| **GET** `/auth/me` | app/api/routes/auth.py:144 | src/services/auth.ts:99 |
| **GET** `/auth/sessions` | app/api/routes/auth.py:157 | src/services/auth.ts:101 |
| **POST** `/auth/sessions/revoke-others` | app/api/routes/auth.py:178 | src/services/auth.ts:102 |
| **DELETE** `/auth/sessions/{session_id}` | app/api/routes/auth.py:193 | src/services/auth.ts:103 |
| **POST** `/auth/email/send-verification` | app/api/routes/auth.py:217 | src/services/auth.ts:105 |
| **POST** `/auth/email/verify` | app/api/routes/auth.py:228 | src/services/auth.ts:106 |
| **POST** `/auth/password/forgot` | app/api/routes/auth.py:237 | src/services/auth.ts:108 |
| **POST** `/auth/password/reset` | app/api/routes/auth.py:248 | src/services/auth.ts:110 |
| **GET** `/phc-application/documents` | app/api/routes/applications.py:84 | src/services/documents.ts:28 |
| **POST** `/phc-application/documents` | app/api/routes/applications.py:89 | src/services/documents.ts:34 |
| **DELETE** `/phc-application/documents/{document_id}` | app/api/routes/applications.py:136 | src/services/documents.ts:37 |
| **GET** `/phc/dashboard` | app/api/routes/phc.py:181 | src/services/phc.ts:187 |
| **GET** `/phc/overview` | app/api/routes/phc.py:233 | src/services/phc.ts:188 |
| **GET** `/phc/queue` | app/api/routes/phc.py:294 | src/services/phc.ts:190 |
| **POST** `/phc/queue/{processed_id}/seen` | app/api/routes/phc.py:336 | src/services/phc.ts:192 |
| **GET** `/phc/worklist` | app/api/routes/phc.py:362 | src/services/phc.ts:194 |
| **POST** `/phc/worklist/citizens/{user_id}/resolve` | app/api/routes/phc.py:453 | src/services/phc.ts:196 |
| **GET** `/phc/citizens` | app/api/routes/phc.py:468 | src/services/phc.ts:200 |
| **POST** `/phc/citizens/{user_id}/readings` | app/api/routes/phc.py:560 | src/services/phc.ts:242 |
| **GET** `/phc/citizens/{user_id}/record` | app/api/routes/phc.py:582 | src/services/phc.ts:202 |
| **GET** `/phc/alerts` | app/api/routes/phc.py:725 | src/services/phc.ts:237 |
| **GET** `/phc/reports` | app/api/routes/phc.py:766 | src/services/phc.ts:203 |
| **GET** `/phc/locality-risk` | app/api/routes/phc.py:838 | no screen uses it yet |
| **GET** `/phc/map` | app/api/routes/phc.py:845 | src/services/phc.ts:204 |
| **POST** `/phc/visits` | app/api/routes/phc.py:858 | src/services/phc.ts:218 |
| **GET** `/phc/visits` | app/api/routes/phc.py:911 | no screen uses it yet |
| **GET** `/phc/citizens/{user_id}/missed-visits` | app/api/routes/phc.py:934 | src/services/phc.ts:252 |

| Endpoint | Handler | Called from |
|---|---|---|
| **GET** `/schedule` | app/api/routes/schedule.py:97 | src/services/phc.ts:219 |
| **PATCH** `/schedule/{visit_id}` | app/api/routes/schedule.py:163 | src/services/phc.ts:224 |
| **GET** `/schedule/all` | app/api/routes/schedule.py:262 | src/services/phc.ts:216 |
| **GET** `/alerts` | app/api/routes/alerts.py:30 | src/services/health.ts:227 |
| **GET** `/alerts/unread-count` | app/api/routes/alerts.py:57 | src/services/health.ts:230 |
| **POST** `/alerts/mark-read` | app/api/routes/alerts.py:62 | src/services/health.ts:232 |
| **GET** `/phc-alerts/unread-count` | app/api/routes/alerts.py:78 | src/services/phc.ts:206 |
| **GET** `/advisories` | app/api/routes/advisories.py:229 | src/services/advisories.ts:61 |
| **GET** `/advisories/targets` | app/api/routes/advisories.py:314 | src/services/advisories.ts:56 |
| **POST** `/advisories/reach` | app/api/routes/advisories.py:349 | src/services/advisories.ts:58 |
| **GET** `/advisories/red-check` | app/api/routes/advisories.py:361 | src/services/advisories.ts:65 |
| **POST** `/advisories` | app/api/routes/advisories.py:368 | src/services/advisories.ts:62 |
| **POST** `/advisories/{advisory_id}/withdraw` | app/api/routes/advisories.py:401 | src/services/advisories.ts:63 |
| **POST** `/advisories/{advisory_id}/feedback` | app/api/routes/advisories.py:416 | src/services/advisories.ts:67 |
| **GET** `/clusters` | app/api/routes/clusters.py:155 | src/services/outbreaks.ts:43 |
| **POST** `/clusters/{cluster_id}/acknowledge` | app/api/routes/clusters.py:169 | src/services/outbreaks.ts:47 |
| **POST** `/clusters/{cluster_id}/feedback` | app/api/routes/clusters.py:185 | src/services/outbreaks.ts:45 |
| **GET** `/settings/preferences` | app/api/routes/settings.py:48 | src/services/account.ts:87 |
| **PATCH** `/settings/preferences` | app/api/routes/settings.py:53 | src/services/account.ts:88 |
| **GET** `/settings/consents` | app/api/routes/settings.py:86 | src/services/account.ts:90 |
| **PATCH** `/settings/consents` | app/api/routes/settings.py:91 | src/services/account.ts:92 |
| **GET** `/settings/consents/history` | app/api/routes/settings.py:104 | src/services/account.ts:94 |
| **GET** `/settings/about` | app/api/routes/settings.py:141 | src/services/account.ts:96 |
| **GET** `/settings/export` | app/api/routes/settings.py:187 | src/services/account.ts:99 |
| **POST** `/settings/password` | app/api/routes/settings.py:279 | src/services/account.ts:109 |
| **POST** `/settings/deactivate` | app/api/routes/settings.py:305 | src/services/account.ts:111 |
| **POST** `/settings/concerns` | app/api/routes/settings.py:349 | src/services/account.ts:115 |
| **GET** `/settings/concerns` | app/api/routes/settings.py:363 | src/services/account.ts:113 |
| **GET** `/settings/concerns/{concern_id}` | app/api/routes/settings.py:375 | no screen uses it yet |
| **GET** `/settings/health-centre` | app/api/routes/settings.py:436 | src/services/account.ts:117 |
| **PATCH** `/settings/health-centre` | app/api/routes/settings.py:443 | src/services/account.ts:118 |
| **GET** `/profile` | app/api/routes/profile.py:116 | src/services/account.ts:120 |
| **PATCH** `/profile` | app/api/routes/profile.py:121 | src/services/account.ts:122 |
| **POST** `/profile/avatar` | app/api/routes/profile.py:174 | src/services/account.ts:123 |

| Endpoint | Handler | Called from |
|---|---|---|
| **DELETE** `/profile/avatar` | app/api/routes/profile.py:183 | src/services/account.ts:128 |
| **POST** `/profile/cover` | app/api/routes/profile.py:191 | src/services/account.ts:123 |
| **DELETE** `/profile/cover` | app/api/routes/profile.py:200 | src/services/account.ts:128 |
| **POST** `/profile/email` | app/api/routes/profile.py:217 | src/services/account.ts:130 |
| **POST** `/profile/email/confirm` | app/api/routes/profile.py:243 | src/services/account.ts:131 |
| **GET** `/assistant/info` | app/api/routes/assistant.py:86 | src/services/assistant.ts:41 |
| **GET** `/assistant/sessions` | app/api/routes/assistant.py:96 | src/services/assistant.ts:42 |
| **GET** `/assistant/sessions/{session_id}` | app/api/routes/assistant.py:108 | src/services/assistant.ts:44 |
| **PATCH** `/assistant/sessions/{session_id}` | app/api/routes/assistant.py:124 | src/services/assistant.ts:45 |
| **DELETE** `/assistant/sessions/{session_id}` | app/api/routes/assistant.py:139 | src/services/assistant.ts:46 |
| **DELETE** `/assistant/sessions` | app/api/routes/assistant.py:148 | no screen uses it yet |
| **POST** `/assistant/ask` | app/api/routes/assistant.py:154 | src/services/assistant.ts:49 |

Handlers link to the first line of the function; the third column links to the line that makes the request. Every `/phc/...` route is protected by the router-level guard `require_catchment_worker`.

# 18 Data model and numbers to know

The tables the worker module reads and writes, then every threshold with the line that defines it.

| Table (model) | What the worker module uses it for | Defined at |
|---|---|---|
| `users` | role (pending_phc → phc_worker), phc_id (the catchment link), area, geo_lat/lng, preferences | app/models/user.py:35 |
| `phc_applications` | employee_id, designation, centre name, status (pending, approved, rejected), reviewer | app/models/phc_application.py:17 |
| `phc_verification_documents` | uploaded proofs, SHA-256, OCR text, tamper score, check status | app/models/phc_verification_document.py:15 |
| `valid_phc_facilities` | the registry of centres, with coordinates | app/models/valid_phc_facility.py:12 |
| `health_records` | readings; `source` watch / phc / self; `recorded_by` and `location` for examinations | app/models/health_record.py:14 |
| `processed_data` | the flag: score, working, and the worker's resolution, reason, note, reviewer | app/models/processed_data.py:15 |
| `scheduled_visits` | visits, kind, length, private notes, who cancelled and why | app/models/scheduled_visit.py:11 |
| `alerts` | the worker's notifications | app/models/alert.py:10 |
| `community_data` | outbreak clusters, with acknowledgement and feedback | app/models/community_data.py:11 |
| `advisories` | what the worker publishes | app/models/advisory.py:11 |
| `concerns` | complaints, with subject and evidence | app/models/concern.py:17 |
| `audit_logs` | view_citizen_record, record_examination, document views | app/models/audit_log.py:10 |

| Setting | Value | Defined at |
|---|---|---|
| 'Gone quiet' after | 14 days without a reading | app/api/routes/phc.py:91 |
| Flags shown per citizen on a card | 10 worst | app/api/routes/phc.py:94 |
| Record timeline | 200 entries (2000 readings read) | app/api/routes/phc.py:92 |
| KPI history | 8 weeks, Monday start, India time | app/api/routes/phc.py:229 |
| Smallest area figure shown | 3 citizens | app/services/localities.py:17 |
| Locality risk levels | 30% high, 15% elevated | app/services/localities.py:28 |
| Examination time | not in the future, at most 30 days back | app/schemas/health.py:12 |
| Symptoms on one reading | 12 | app/schemas/health.py:11 |
| Book a visit | at most 90 days ahead; one open visit per citizen | app/api/routes/schedule.py:25 |
| Cancel or move a visit | not within 8 hours; cancel needs a 10+ character reason | app/api/routes/schedule.py:30 |
| Visit 'no outcome' grace | 4 hours | app/services/visit_state.py:23 |
| Complaint about missed visits | 3 unexplained in 90 days | app/services/complaints.py:22 |
| Proof documents | up to 6, 5 MB each, JPEG, PNG or PDF | app/api/routes/applications.py:23 |

| Setting | Value | Defined at |
| --- | --- | --- |
| Outbreak clustering | 14-day window, 56-day baseline, p below 0.01, every 6 hours | app/services/outbreaks.py:31 |
| Red advisory | 10+ cases in a flagged outbreak, or 15+ high-alert citizens and 5% of the area | app/api/routes/advisories.py:155 |
| Badge refresh | every 30 seconds | src/components/app/AppShell.tsx:47 |

# 19 Questions across the whole worker module

Short answers with the place to open if the examiner says 'show me'.

### Questions you may be asked

**Q. What exactly can a health worker see?**
Only citizens whose centre is theirs and who have not withdrawn worker access (`catchment_citizens`). Within that: profile, readings, flags, visits and alerts. Area-level pages add only people who opted into community monitoring.

**Q. How do you stop a worker reading another centre's citizens?**
Every query starts from `catchment_ids(worker)`, the guard is on the whole `/phc` router, and a citizen outside the catchment returns the same 404 as an unknown id (`get_catchment_citizen`).

**Q. How is a worker accountable?**
Viewing a record, recording an examination and each document the admin opens write `audit_logs` rows; every resolution stores who, when, the reason and the note on the flag itself.

**Q. How does someone become a worker?**
They apply as `pending_phc` with an employee ID and proof; automatic checks help the administrator; only an administrator approves and links the centre. Until then the account has no access to health information.

**Q. What is a flag and is it a diagnosis?**
A flag says a reading is unusual for that person or outside a safe range. It is a prompt to follow up. The worker resolves or dismisses with a reason; nothing is concluded automatically.

**Q. Why group alerts by citizen?**
One person with five flags is one task. The worklist ranks people by their worst flag and lets the worker clear all or one.

**Q. What prevents a visit being booked twice or cancelled at the last minute?**
One open visit per citizen, a booking horizon of 90 days, and a server-side 8-hour cut-off with a written reason for cancellation (`check_cutoff`).

**Q. How is privacy kept in reports and maps?**
Only opted-in citizens are counted, and any area under three people is merged into 'Other localities' (`suppress`).

**Q. Why are weeks and days counted in India time?**
So a flag raised late in the evening lands on the correct local day; SQL converts to the local zone before bucketing.

**Q. What happens if a worker's approval is withdrawn?**
`revoke` returns them to `pending_phc`, removes `phc_id`, cancels their booked visits and informs the citizens.

**Q. What would you test first?**
The catchment boundary (foreign ids), consent withdrawal hiding a citizen, the resolve and dismiss rules, the visit cut-off, area-figure suppression and the examination's consent check. Test files in `backend/tests` are named by feature (test_modules, test_consents, test_notifications, test_watch, test_onboarding).